Computer Networks Phase 1 Report

Private DNS and HTTPS Load Balancing on a Local Network

Review 1 submission | Evidence collected 4 and 5 October 2026

We built a local service in which a private DNS name resolves to an nginx HTTPS edge, which forwards requests to two Python backends. The tests demonstrate name resolution, trusted TLS, backend distribution, HTTP cache revalidation, and the effects of five controlled failures.

Member	Roll number	Machine and responsibility
Aradhya Tiwari	2401010093	Mac 1: DNS, client tests and packet capture
Chaitanya Singh	2401010133	Mac 2: nginx edge, TLS and load balancing
Vedant Satbhai	2401010500	Windows: Backend A and firewall
Anuj Upadhyay	2401010086	Mac 4: Backend B and DNS client

Scope and result

This submission covers Phase 1 only. Both backends served successful responses through the private domain, and all 12 directional LAN ping checks passed. The implementation was shut down after evidence collection; the saved results describe that demonstration session, not a continuously running service.

One machine uses Windows instead of macOS. The network roles and protocols are unchanged; this hardware variation should be disclosed at review. The both-backends-unavailable demonstration returned HTTP 504 rather than the assignment table's expected 502; the report explains the recorded upstream errors.

Reading the submission

The next sections cover architecture, implementation, results, failures and protocol analysis. The evidence appendix embeds original screenshots. Evidence/index.csv maps each figure to its original file. Guides contains restart, demonstration and viva instructions; Code and Configuration contain the reproducible implementation.

Architecture and addressing

Machine	IPv4	Active interface	Active MAC
Aradhya	10.7.15.68	en0	92:a8:ba:c6:0f:f8
Chaitanya	10.7.22.151	en0	82:d4:f7:24:36:5a
Vedant	10.7.2.89	Wi-Fi	14:b5:cd:7c:9e:b1
Anuj	10.7.6.93	en0	a2:c0:5d:98:c7:64

All four addresses are on 10.7.0.0/19, with mask 255.255.224.0 and gateway 10.7.0.1. These were DHCP addresses at capture time. The active Mac interface addresses differ from the hardware Wi-Fi IDs reported by networksetup; the inventory uses the active ifconfig addresses.

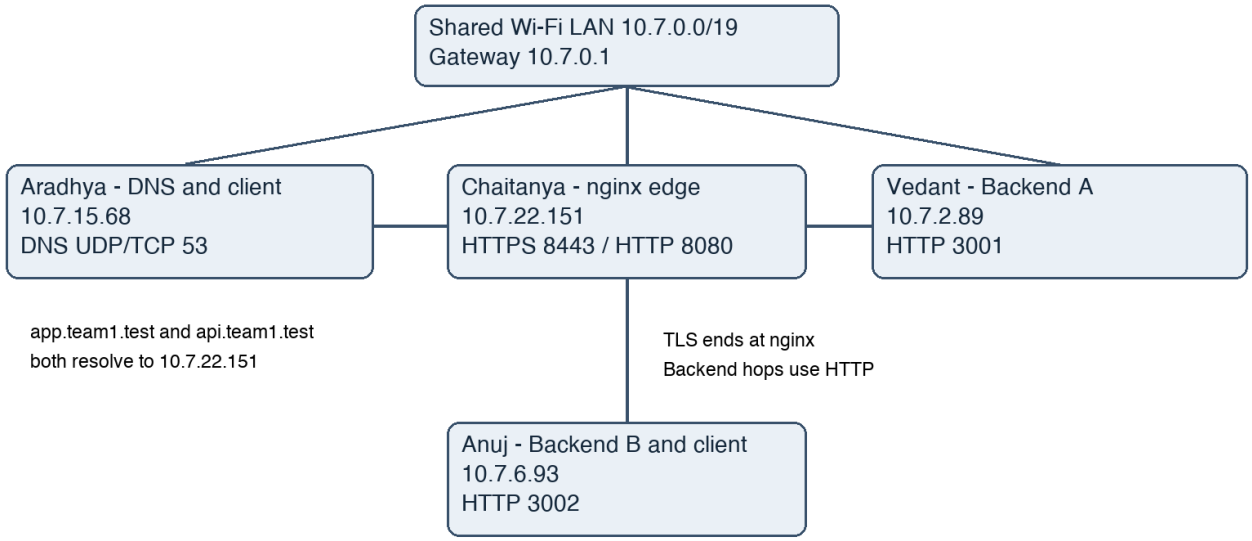
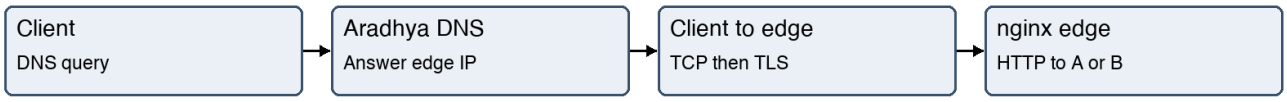


Figure A. Logical topology and service ports. Chaitanya and Anuj also act as clients of Aradhya's DNS. The router is the default gateway; same-subnet communication does not inherently require a routed hop.

Protocol request flow



Application: DNS lookup -> HTTPS request -> HTTP backend response

Transport: UDP 53 -> TCP 8443 -> TCP 3001 or 3002

Network and link: IPv4 addressing over the shared Wi-Fi LAN

Response: Backend -> nginx -> encrypted TLS response -> client

Request flow and implementation

Step	Protocol and action
1	Client asks 10.7.15.68 for app.team1.test using DNS, normally UDP 53. TCP DNS was also tested.
2	DNS returns 10.7.22.151. Client opens TCP from an ephemeral port to edge port 8443.
3	Client and nginx negotiate TLS and validate the certificate for the domain.
4	Client sends HTTP inside TLS. nginx selects an available upstream.
5	nginx sends HTTP/1.1 to A on 3001 or B on 3002. The response includes X-Backend.
6	nginx returns the response through the encrypted client connection, with X-Edge: Chaitanya.

DNS and TLS configuration

dnsmasq listens on 127.0.0.1 and 10.7.15.68, serves app.team1.test and api.team1.test with a 30-second TTL, and forwards other names to 8.8.8.8. Chaitanya and Anuj used this resolver through their normal network DNS settings. The .test namespace avoids .local multicast-DNS conflicts.

nginx terminates TLS on 8443 and redirects HTTP 8080 to HTTPS. HTTP/2 is enabled; backend proxying uses HTTP/1.1. A self-signed project certificate covers both names and was explicitly trusted on the client Macs. Final HTTPS checks did not use curl -k. The certificate is valid from 4 October to 3 November 2026; replace and re-trust it for a later demo.

Backend behavior and balancing

Both Python applications use ThreadingHTTPServer, bind to 0.0.0.0, and implement GET and HEAD. Routes / and /api/status return JSON with the backend identity and no-store. Route /cache returns identical content on both servers, public max-age=60 and a SHA-256 ETag. A matching If-None-Match produces 304 without a body.

nginx uses two unweighted upstreams, max_fails=1 and fail_timeout=5s. It retries errors and timeouts with at most two upstream attempts. This is passive failure handling, not an active health-check service. Recorded repeated requests alternated A and B; exact alternation is not guaranteed during failures or concurrent traffic.

Verification results

Requirement	Observed result	Evidence
LAN	Every directed pair: 3 sent, 3 received, 0% loss	Figures E05 to E09
DNS clients	Both names resolved through 10.7.15.68 on Chaitanya and Anuj	Client setup screenshots
Backend routes	GET / returned 200 from A, B and HTTPS edge	root-endpoint-final-check.json
HTTPS	Trusted domain requests returned HTTP/1.1 and HTTP/2 200	Client and load-balancing screenshots
Balancing	Six requests returned A B A B A B, all 200	Figure E17 and access.log
Cache	200 from A, matching ETag conditional request returned 304 from B	Figures E18 and E19
Protocol flow	DNS, TCP, TLS and encrypted data in a saved capture	Figures E13 to E16 and pcapng

Ping matrix

Source to destination	Aradhya	Chaitanya	Vedant	Anuj
Aradhya	Self	3/3	3/3	3/3
Chaitanya	3/3	Self	3/3	3/3
Vedant	3/3	3/3	Self	3/3
Anuj	3/3	3/3	3/3	Self

Windows initially did not answer ping. After a scoped inbound ICMPv4 echo rule was created, the Mac-to-Windows check passed. HTTP access uses a separate TCP 3001 firewall rule. A prior explicit Python block rule was narrowed to exclude 3001. The additional unrestricted TCP 3001 allow rule supplied later is recorded in the firewall notes for reproducibility and cleanup.

Cache interpretation

max-age=60 allows a capable client to reuse a fresh response for 60 seconds without contacting the origin. Our demonstrated test is conditional revalidation: curl explicitly sent If-None-Match, and the server returned 304. curl did not automatically create a browser-like cache. A full new request returns the body with 200; a 304 confirms that the stored representation can be reused.

Failure demonstrations and recovery

Scenario	Observation	Explanation and recovery
Wrong client DNS	Lookup timed out; ping to edge succeeded	Resolver set to 10.7.2.89. Restoring 10.7.15.68 restored lookup.
Wrong DNS record	NOERROR answer 10.7.2.89; HTTPS connect timeout	DNS can return a valid but wrong destination. Restored record returned 10.7.22.151 and HTTPS 200.
Backend A stopped	Client received 200 from B	nginx retried the surviving upstream. Log has upstream_status=504, 200.
Both unavailable	Client received 504	B refused connection, then A timed out. Restarting backends restored 200 and A/B distribution.
Wrong client port	DNS succeeded; TCP 8444 refused	Correct host, wrong service port. Returning to 8443 restored HTTP 200.

Why the both backend test returned 504

The assignment lists 502 Bad Gateway for both backends stopped. Our actual response was 504 Gateway Time-out. At 23:31:44 IST, nginx request *70 encountered connection refusal from 10.7.6.93:3002. At 23:31:47 it timed out connecting to 10.7.2.89:3001. The access log records overall status=504 and upstream_status=502, 504. We retain this observed result rather than relabelling it as 502. Both demonstrate an upstream application failure while the edge remains reachable; faculty should be shown this distinction.

The logs establish upstream unavailability, not the process state by themselves. Stopping the servers was part of the coordinated test procedure. The timeout alone does not identify a specific firewall or operating-system cause.

Troubleshooting history

Early screenshots include installation output, DNS timeouts and failed Windows pings. They remain in Evidence/Originals as history and are not used as successful final checks. An initial wrong-resolver attempt was inconclusive because its first lookup still used the correct resolver; it was repeated with the configured resolver printed and successful IP pings alongside the lookup failure.

Packet analysis and protocol layers

Frames	Meaning
1 and 2	DNS A query and response for app.team1.test -> 10.7.22.151; UDP client port 60078 and DNS port 53.
3 to 5	TCP SYN, SYN-ACK, ACK: client 10.7.15.68:56700 to edge 10.7.22.151:8443. Relative Seq/Ack: 0; 0/1; 1/1.
6	TLS 1.2 ClientHello with SNI app.team1.test.
9	ServerHello, Certificate, ServerKeyExchange and ServerHelloDone.
11 and 12	Client key exchange, ChangeCipherSpec and encrypted handshake records.
16 and 18	Encrypted application data in the two directions.

The main capture combines loopback and Wi-Fi traffic. DNS appears on loopback with identical source and destination IPs because Aradhya acted as both DNS server and client. The HTTPS connection crosses Wi-Fi to Chaitanya. Separate peer DNS evidence proves DNS access from another Mac. TLS 1.2 was deliberately selected for this trace so the certificate message is visible; the edge also permits TLS 1.3.

TCP acknowledges byte sequence space, retransmits when needed, and uses advertised windows for flow control. SYN consumes one sequence number, explaining the initial acknowledgement of 1. These packets illustrate the handshake and counters; they are not a controlled packet-loss experiment.

Layer	Project examples
Application	DNS and HTTP; REST is an application interface style
TLS placement	TLS operates above TCP and below HTTP; often mapped to OSI session/presentation functions
Transport	UDP/TCP DNS and TCP HTTPS; source and destination ports
Network	IPv4 addresses and local subnet
Link	Wi-Fi LAN; Ethernet-style capture headers and active MAC addresses

HTTP headers are visible in curl after TLS decryption at the endpoint. They are not readable as plaintext HTTP in this encrypted packet capture. The edge-to-backend HTTP hop is not encrypted by this design.

Review preparation and references

Phase 1 demonstration order

Show topology and inventory, then the ping matrix. Resolve the domain normally from Chaitanya or Anuj. Open the service through trusted HTTPS. Repeat requests to show both backend headers. Open the saved capture and explain DNS, TCP, TLS and encrypted data. Demonstrate the cache headers and 304. Present all five failure-and-recovery pairs and explain the affected layer. Each member should be able to explain the complete request flow.

Local roles and cloud equivalents

dnsmasq plays the local name-directory role analogous to managed DNS. nginx resembles an application load balancer with TLS termination. The two Python processes resemble separate application instances. This comparison is conceptual: no cloud hosting, managed health checking or cloud resilience was implemented.

Boundaries

Phase 2 backup DNS, service isolation extensions, DNS cutover and high availability are outside this submission. HTTP/3 and email protocols are explanation-only topics in the brief. A live demonstration is required; a recorded video should be supplied only if separately requested by faculty. The hardware variation and 504 outcome must remain visible in the review discussion.

References and evidence provenance

1. CN_Project_Doc.pdf: Phase 1 tasks in Section 6, failure scenarios in Section 6.3, deliverables in Section 9 and Review 1 marks in Section 10. Provided faculty brief, pages 4 to 8 and 11 to 12.

2. Team screenshots collected during the 4 to 5 October 2026 session, embedded in the following appendix and retained unchanged in Evidence/Originals.

3. Chaitanya's deployed nginx.conf, access.log and error.log, supplied after the demonstration. Original configuration uses his absolute filesystem paths; Code/For_Chaitanya contains the generator for relocation.

4. Original packet capture phase1-protocol-flow.pcapng and saved terminal exports in Evidence. The protocol capture is filtered to the project DNS name and HTTPS port. The separate peer DNS trace is identified as historical diagnostic evidence.

Evidence 1 Aradhya LAN inventory

IPv4, subnet, gateway, en0 and active MAC address.

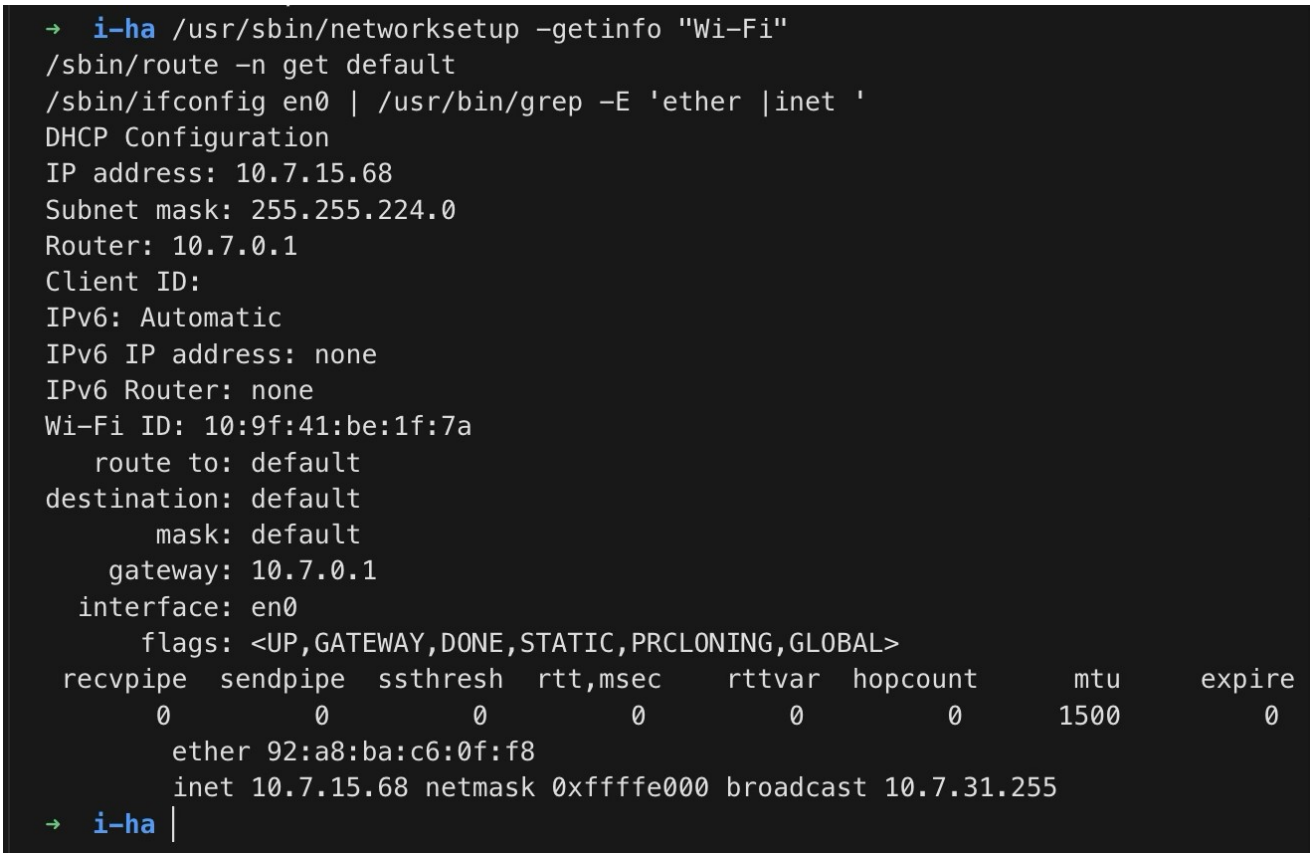


Figure E01. IPv4, subnet, gateway, en0 and active MAC address.

Evidence 2 Chaitanya LAN inventory

Edge address and active en0 MAC. Original full-resolution image is included.

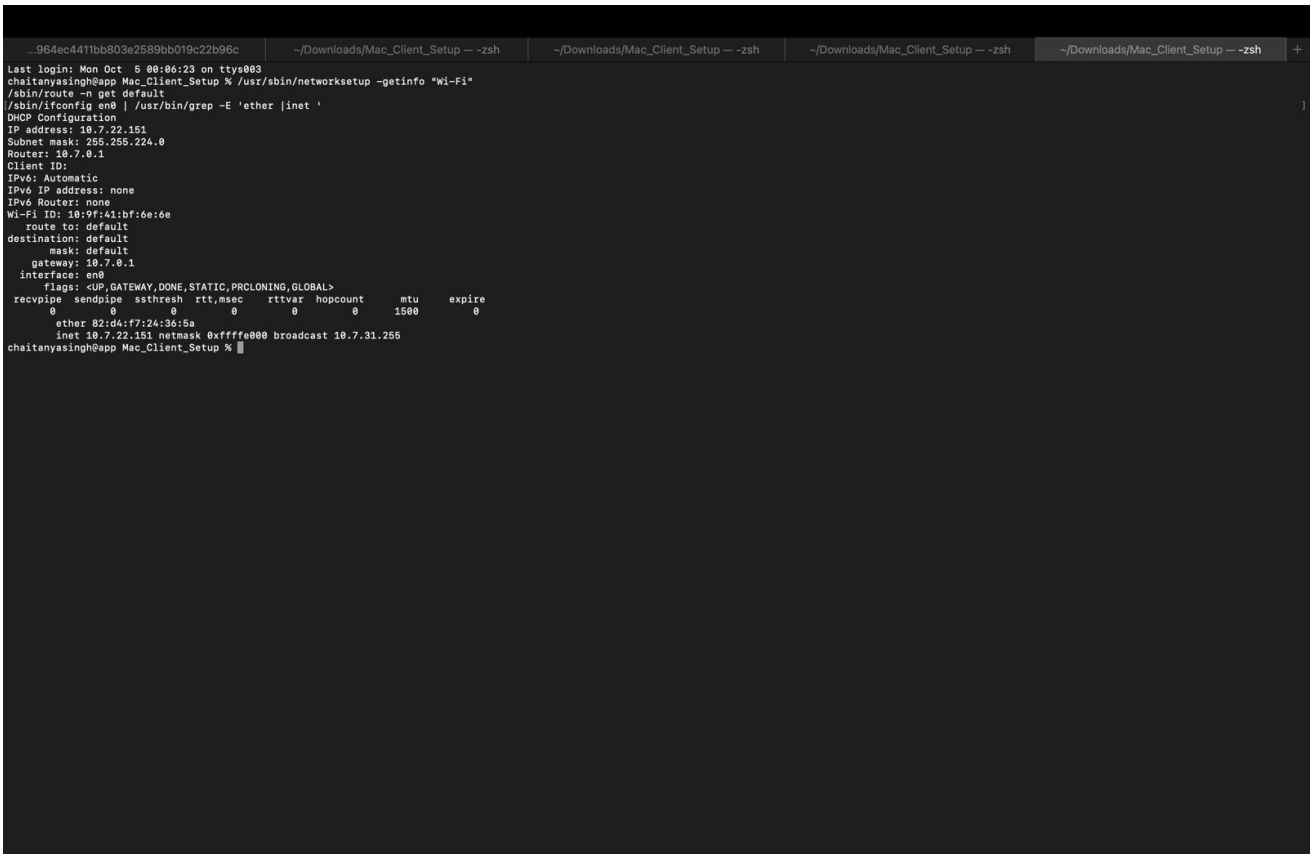


Figure E02. Edge address and active en0 MAC. Original full-resolution image is included.

Evidence 3 Anuj LAN inventory

Backend B address, subnet, gateway and active MAC.

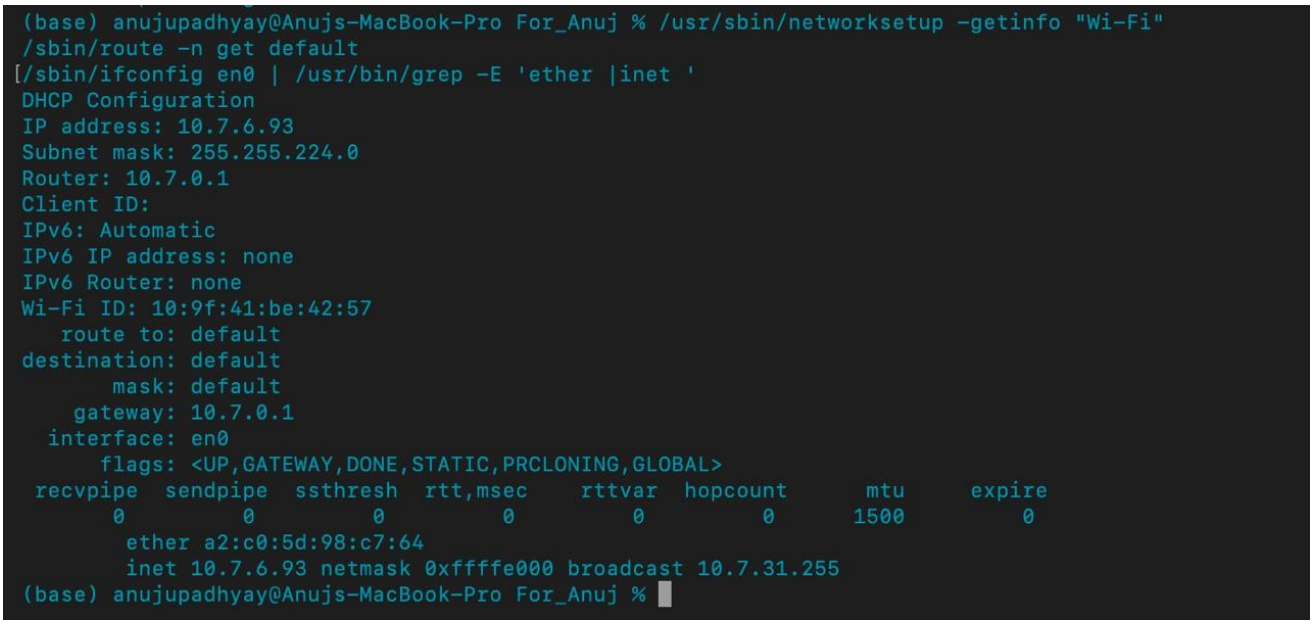


Figure E03. Backend B address, subnet, gateway and active MAC.

Evidence 4 Vedant LAN inventory

Windows Wi-Fi address, /19 prefix, gateway and MAC.

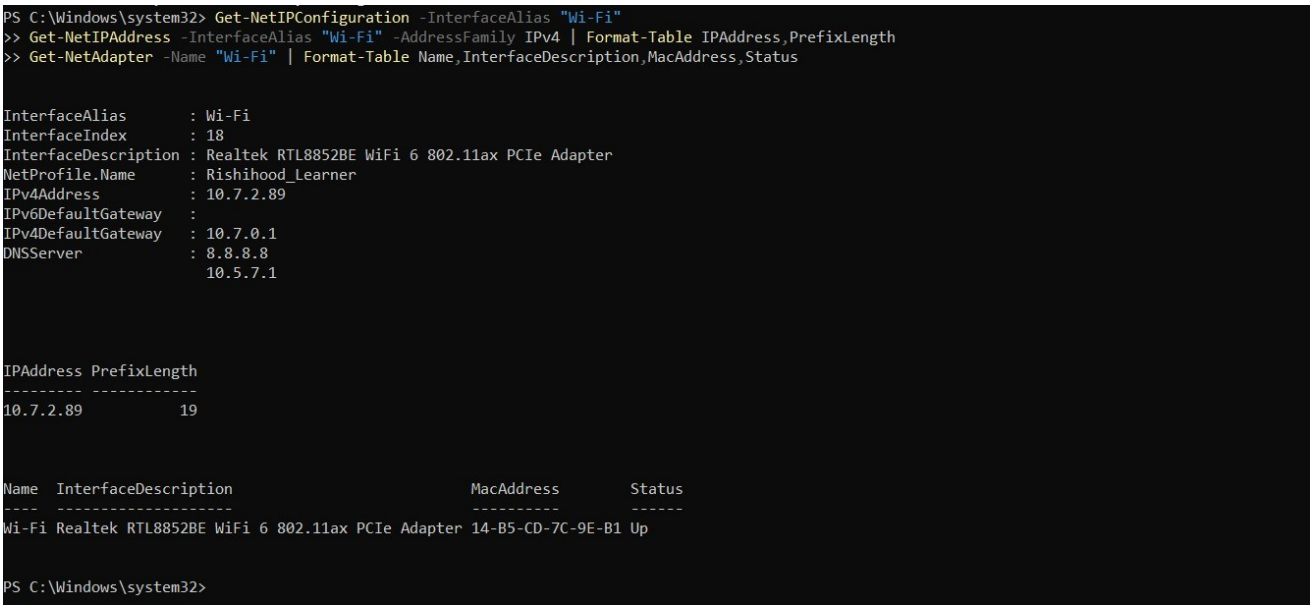


Figure E04. Windows Wi-Fi address, /19 prefix, gateway and MAC.

Evidence 5 Aradhya ping checks

Chaitanya and Anuj pass; the Windows failure shown here is superseded by the next figure.

```
→  i-ha /sbin/ping -c 3 10.7.22.151
/sbin/ping -c 3 10.7.2.89
/sbin/ping -c 3 10.7.6.93
PING 10.7.22.151 (10.7.22.151): 56 data bytes
64 bytes from 10.7.22.151: icmp_seq=0 ttl=32 time=130.593 ms
64 bytes from 10.7.22.151: icmp_seq=1 ttl=32 time=53.145 ms
64 bytes from 10.7.22.151: icmp_seq=2 ttl=32 time=892.450 ms

--- 10.7.22.151 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 53.145/358.729/892.450/378.720 ms
PING 10.7.2.89 (10.7.2.89): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1

--- 10.7.2.89 ping statistics ---
3 packets transmitted, 0 packets received, 100.0% packet loss
PING 10.7.6.93 (10.7.6.93): 56 data bytes
64 bytes from 10.7.6.93: icmp_seq=0 ttl=64 time=11.493 ms
64 bytes from 10.7.6.93: icmp_seq=1 ttl=64 time=64.759 ms
64 bytes from 10.7.6.93: icmp_seq=2 ttl=64 time=6.057 ms

--- 10.7.6.93 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 6.057/27.436/64.759/26.484 ms
→  i-ha
```

Figure E05. Chaitanya and Anuj pass; the Windows failure shown here is superseded by the next figure.

Evidence 6 Windows ping recovery

Aradhya to Vedant passes after the ICMP rule; 3 of 3 replies.

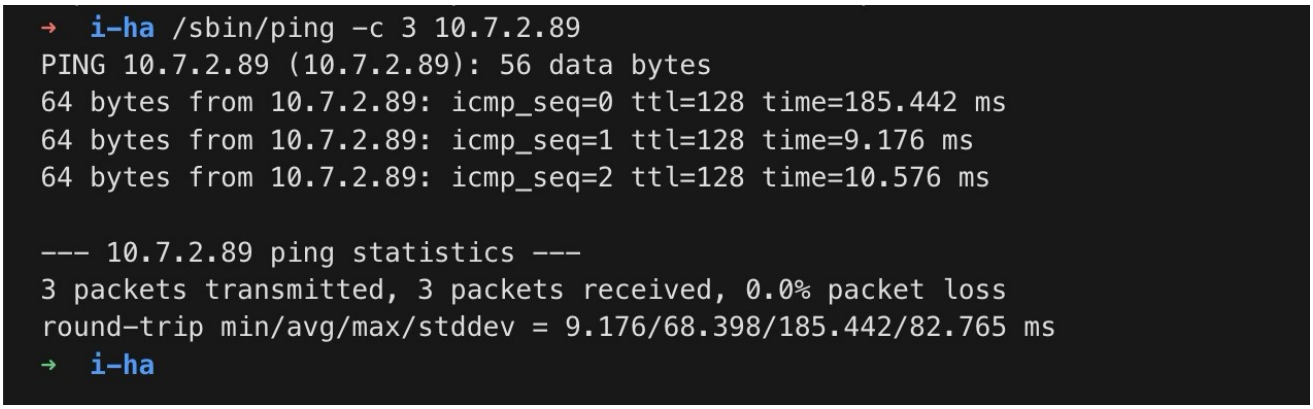


Figure E06. Aradhya to Vedant passes after the ICMP rule; 3 of 3 replies.

Evidence 7 Chaitanya ping checks

All three peer destinations respond with 0% loss.

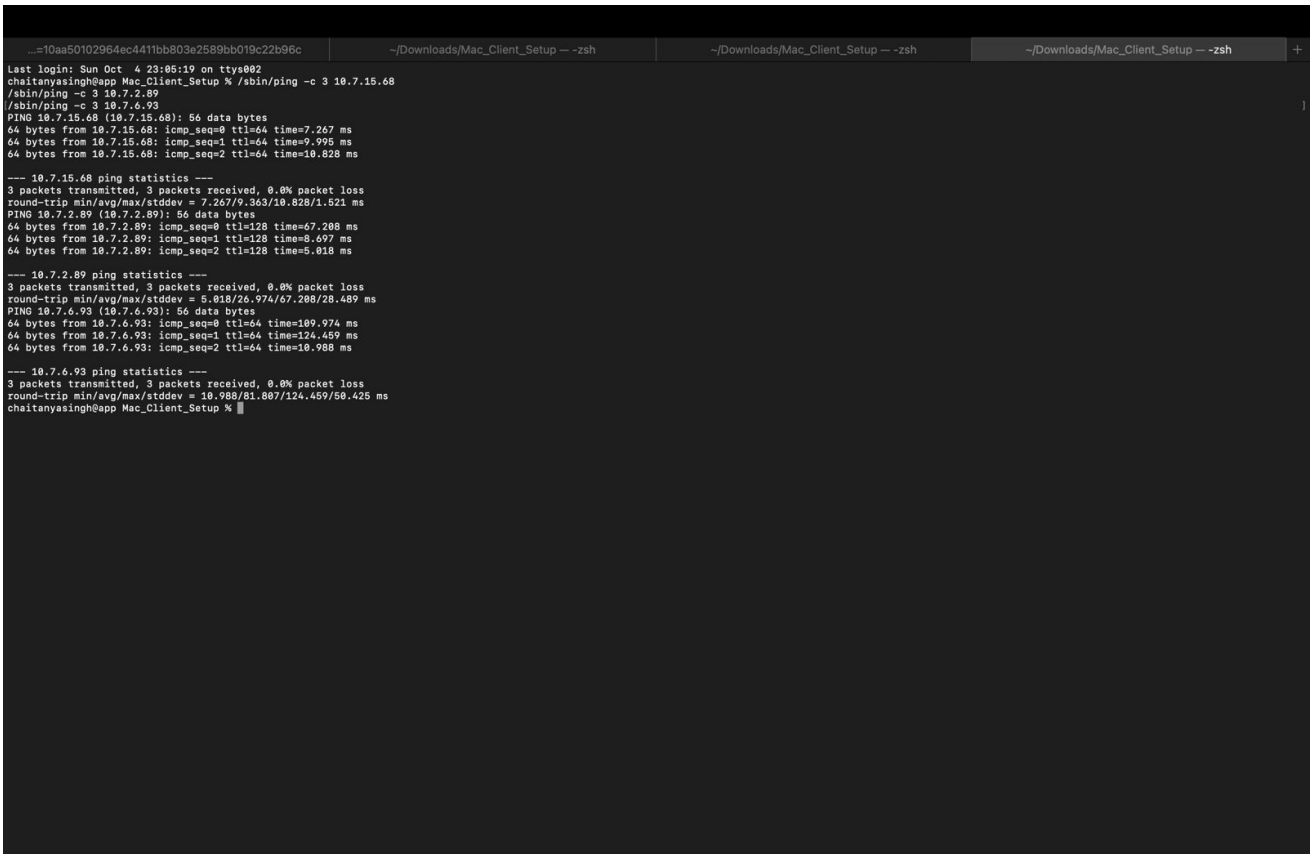


Figure E07. All three peer destinations respond with 0% loss.

Evidence 8 Anuj ping checks

All three peer destinations respond with 0% loss.

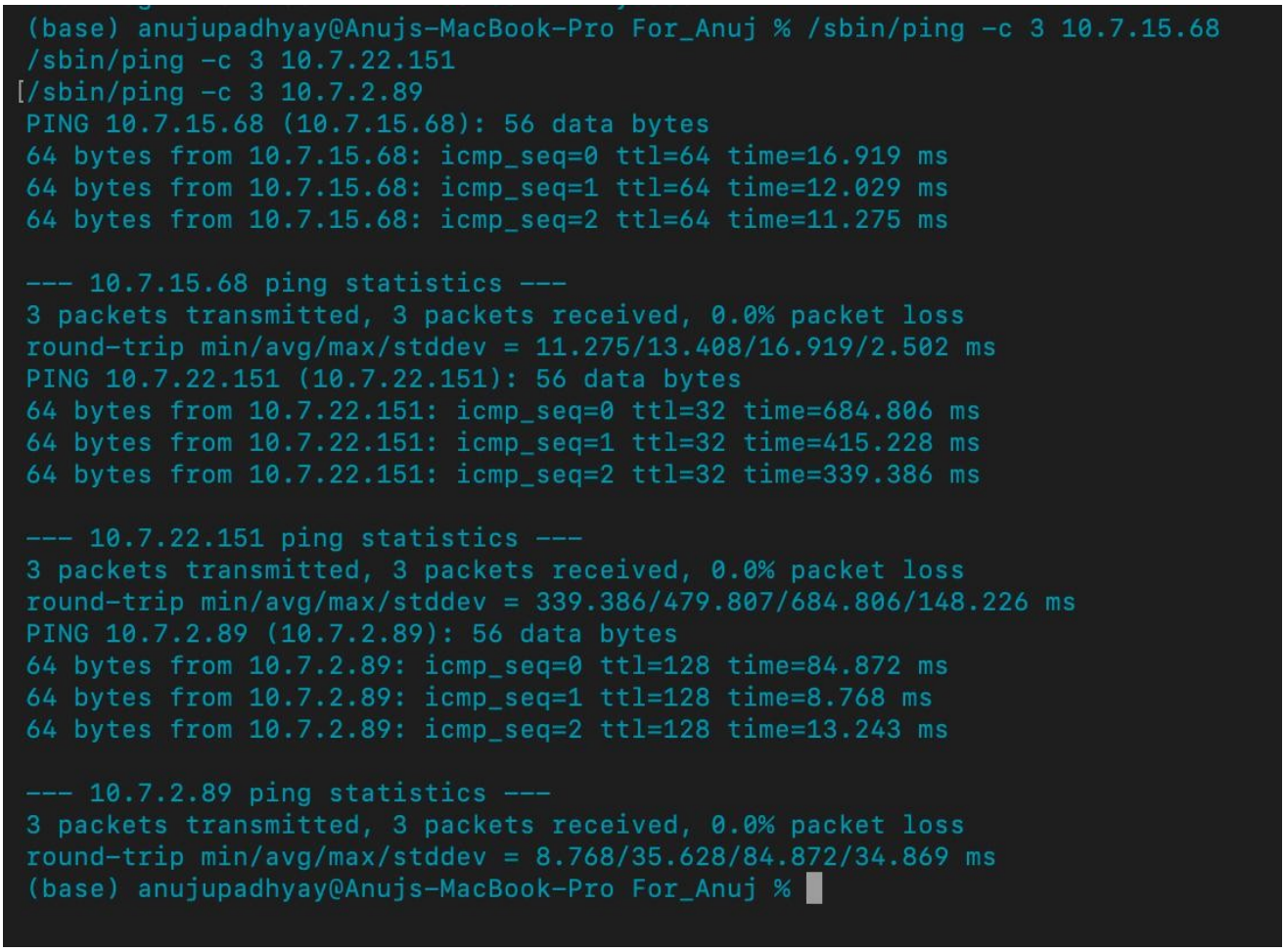


Figure E08. All three peer destinations respond with 0% loss.

Evidence 9 Vedant ping checks

All three Macs respond with 0% loss.

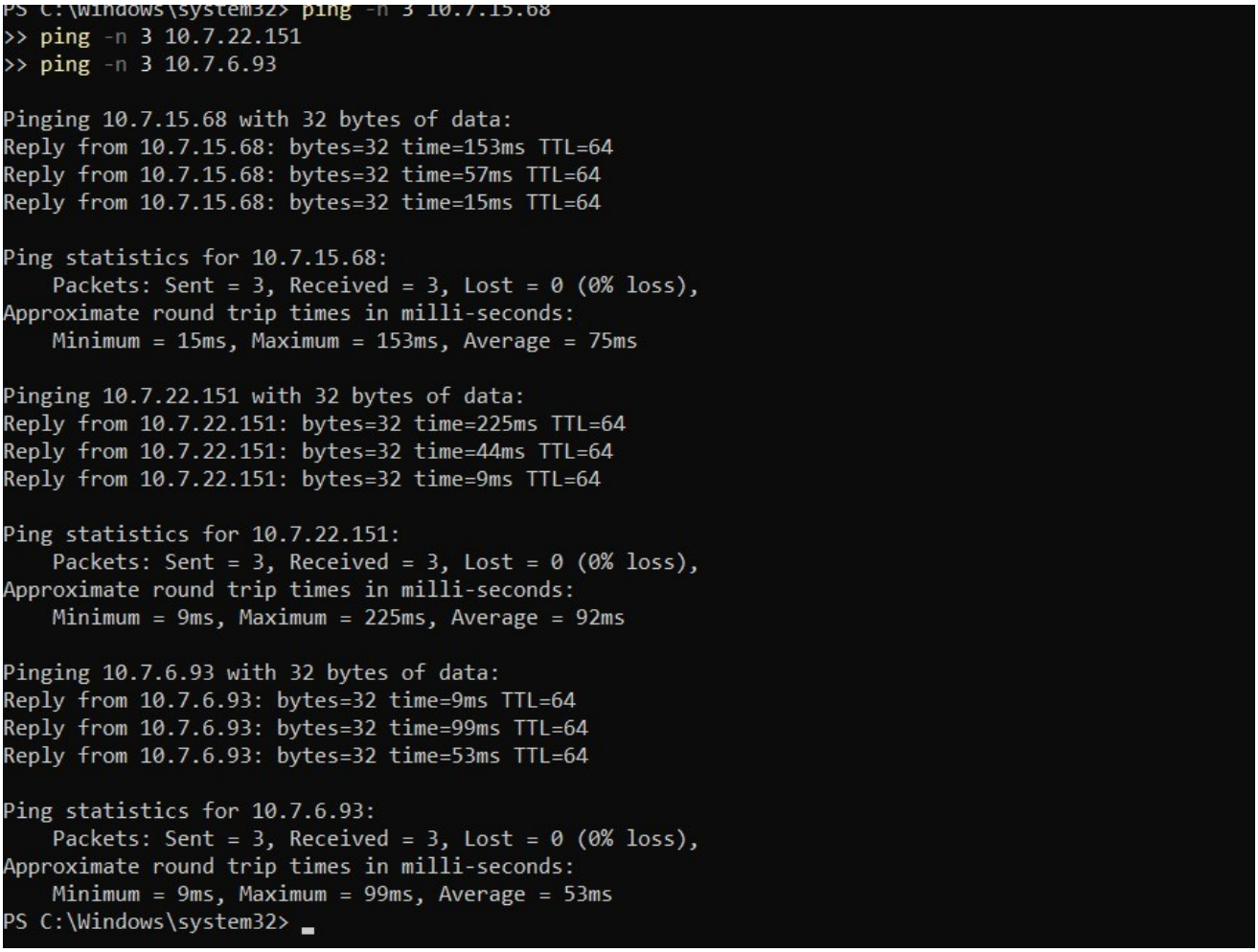


Figure E09. All three Macs respond with 0% loss.

Evidence 10 Aradhya DNS and HTTPS

Normal app/api DNS, native resolver and trusted HTTPS 200.

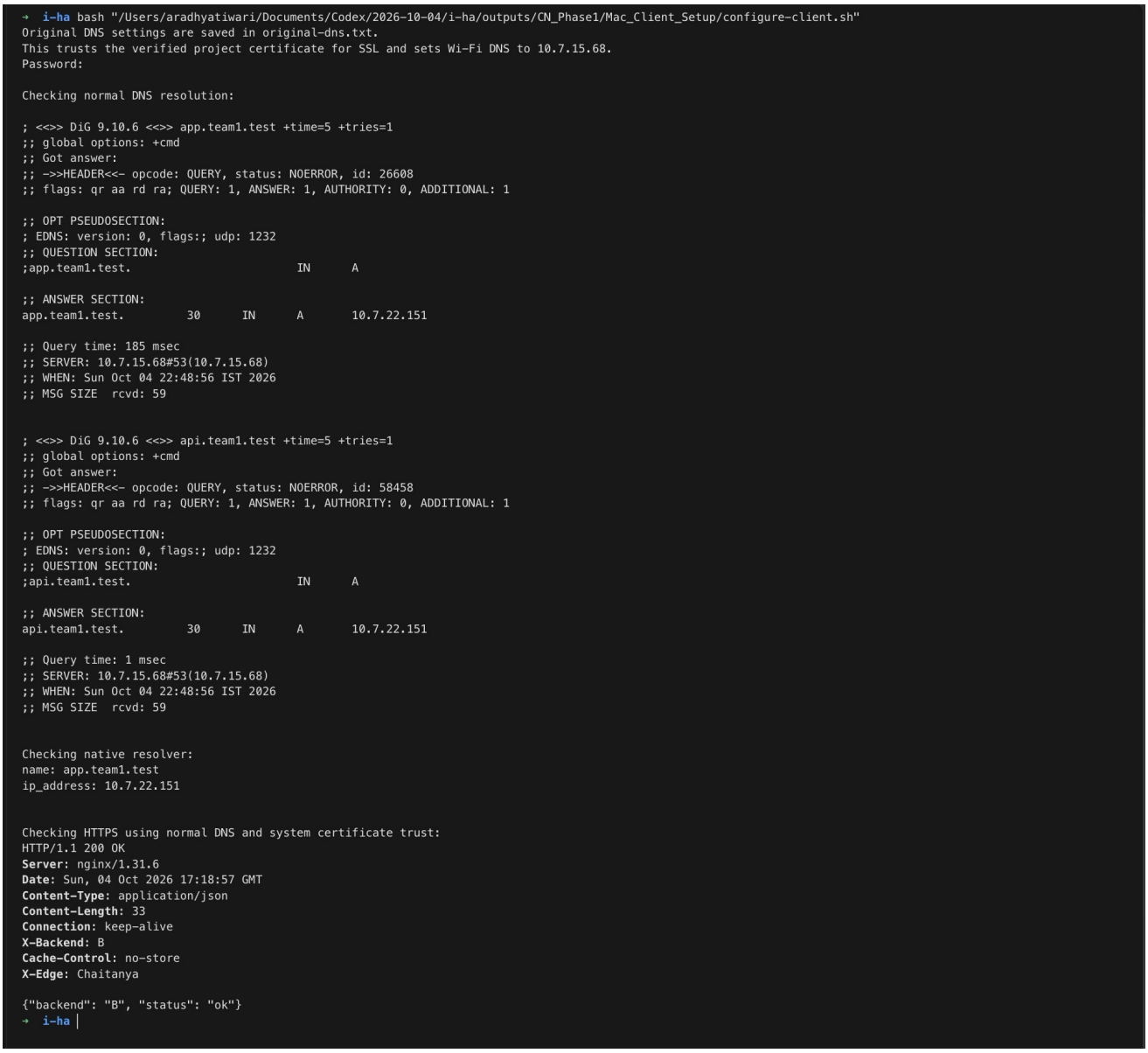


Figure E10. Normal app/api DNS, native resolver and trusted HTTPS 200.

Evidence 11 Chaitanya DNS client

Both names resolve through Aradhya; normal trusted HTTPS succeeds.

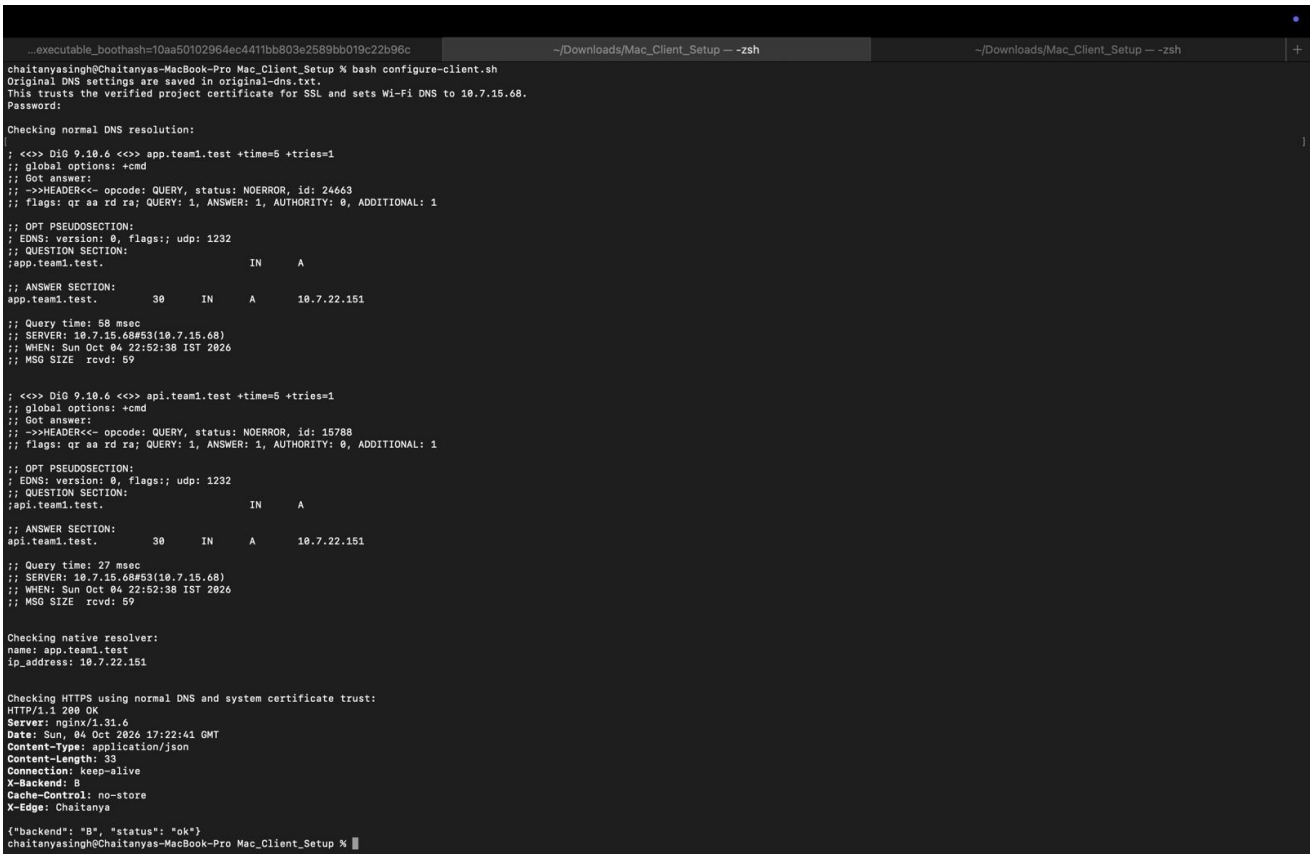


Figure E11. Both names resolve through Aradhya; normal trusted HTTPS succeeds.

Evidence 12 Anuj DNS client

Both names resolve through Aradhya; trusted HTTPS succeeds. Use original image for detailed inspection.

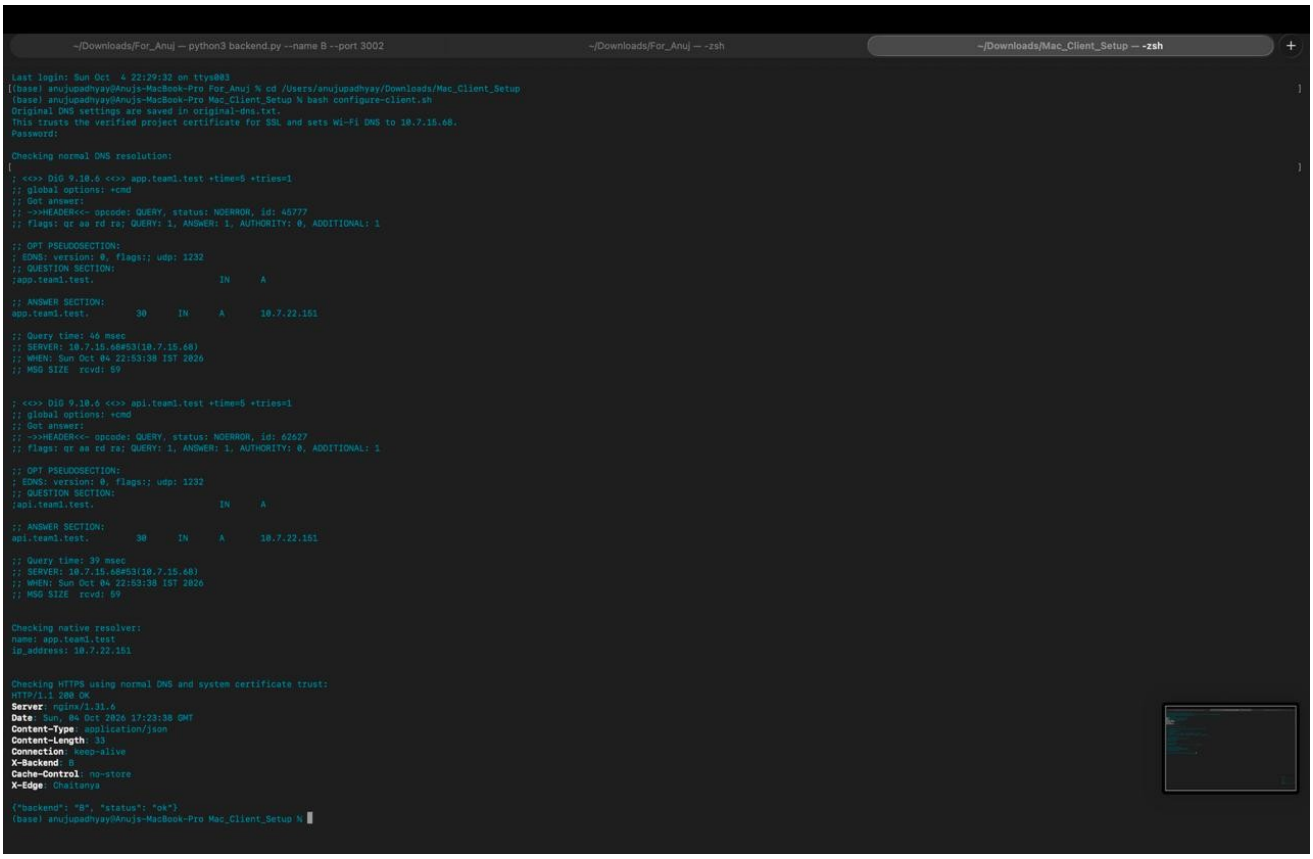


Figure E12. Both names resolve through Aradhya; trusted HTTPS succeeds. Use original image for detailed inspection.

Evidence 13 DNS response details

A record maps app.team1.test to 10.7.22.151 with TTL 30 seconds.

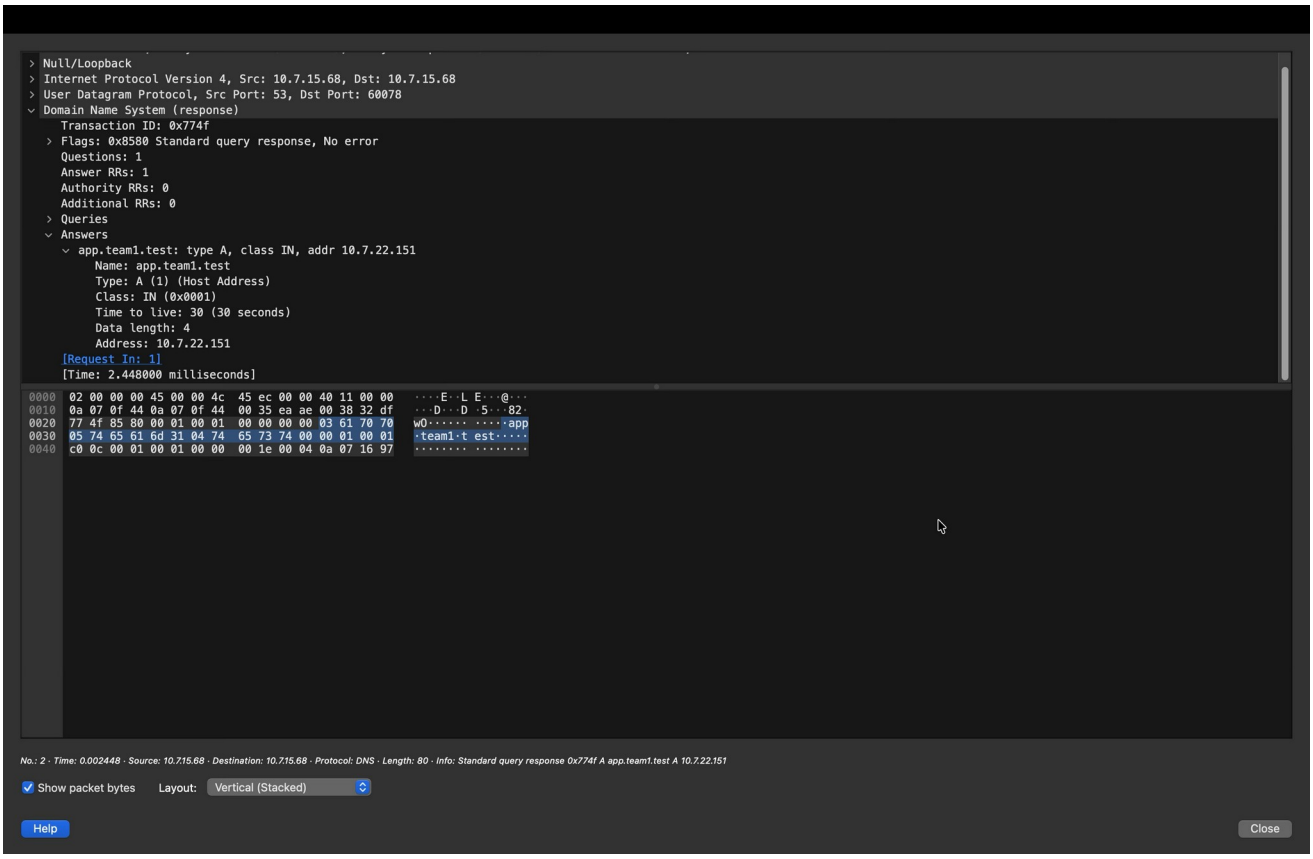


Figure E13. A record maps app.team1.test to 10.7.22.151 with TTL 30 seconds.

Evidence 14 TCP three way handshake

Frames 3 to 5 show SYN, SYN-ACK, ACK and ports 56700 and 8443.

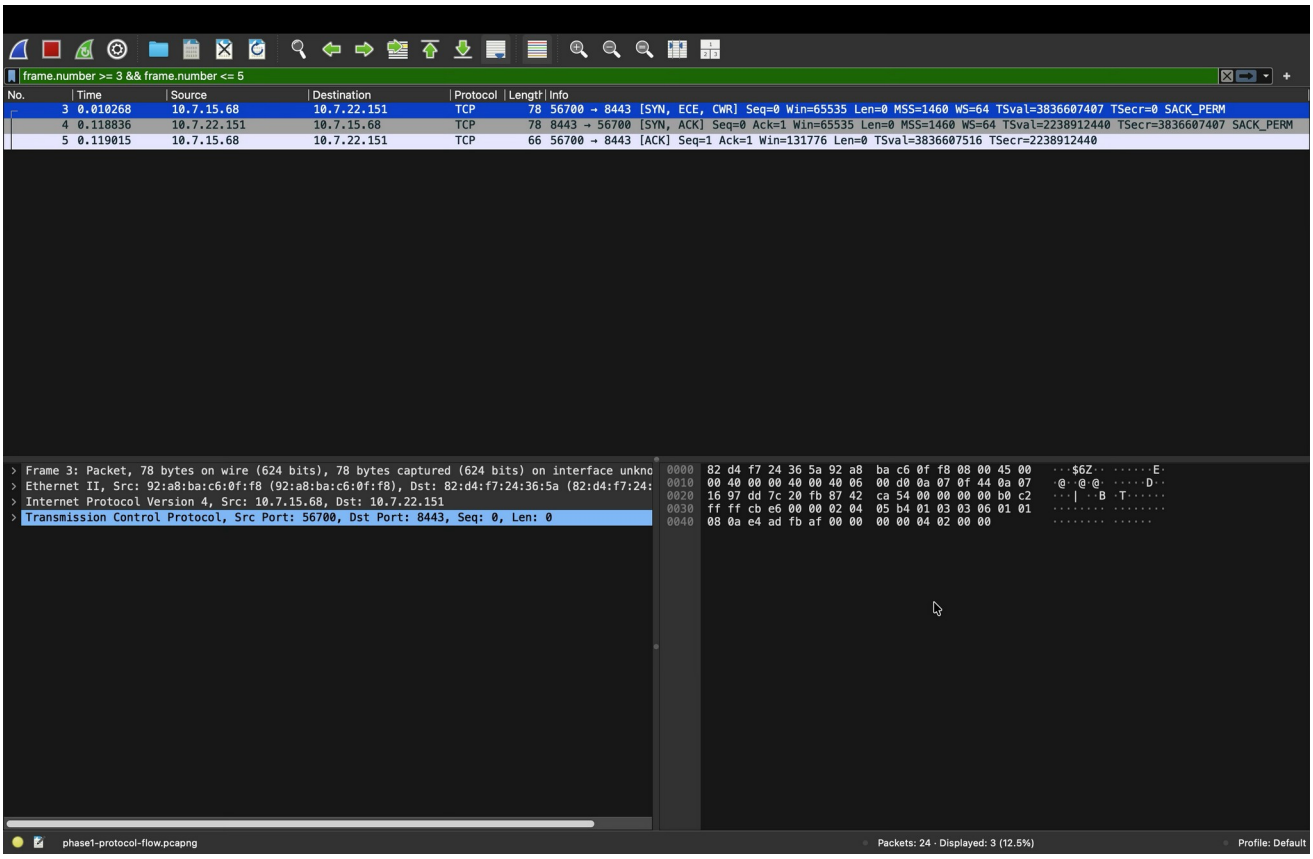


Figure E14. Frames 3 to 5 show SYN, SYN-ACK, ACK and ports 56700 and 8443.

Evidence 15 TLS handshake

ClientHello, server certificate and key exchange, followed by ChangeCipherSpec.

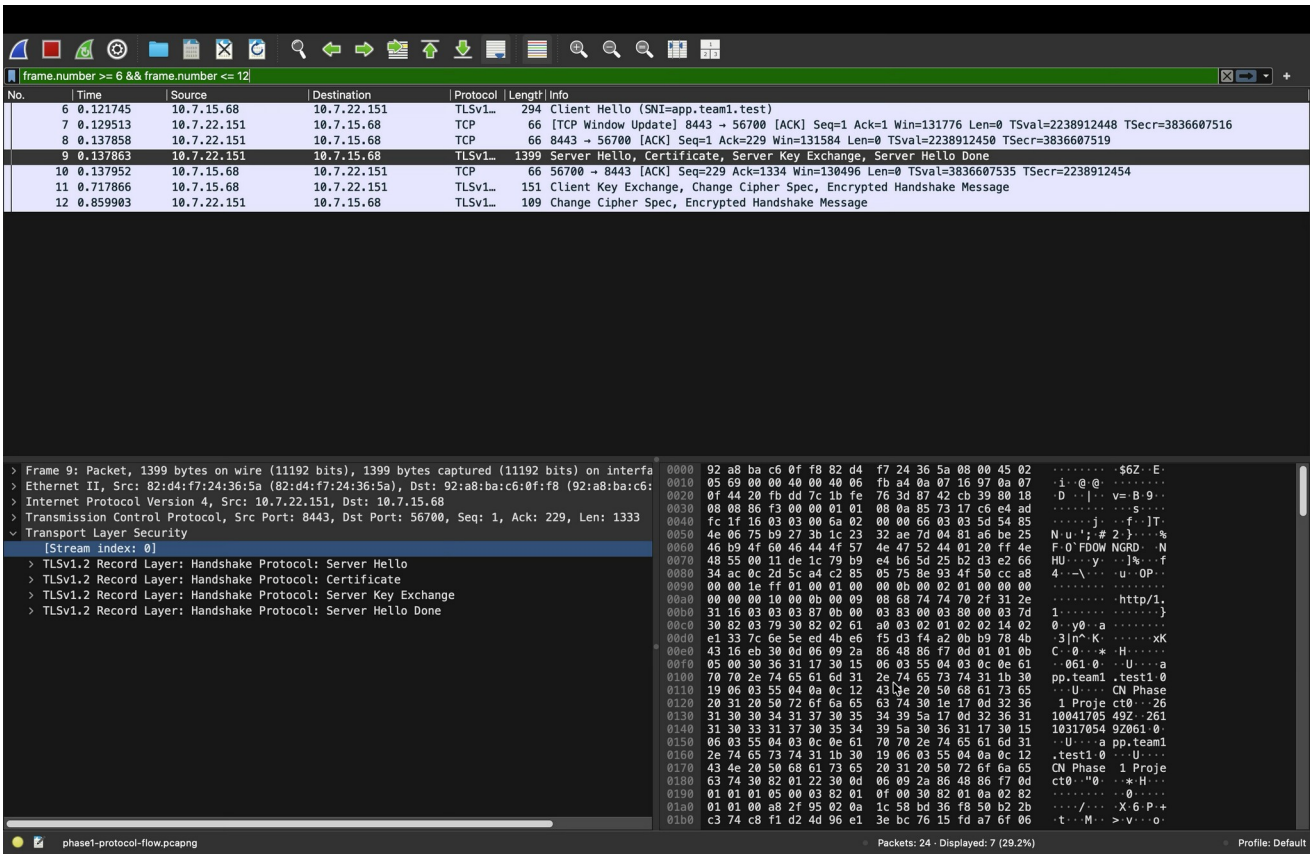


Figure E15. ClientHello, server certificate and key exchange, followed by ChangeCipherSpec.

Evidence 16 Encrypted application data

TLS record content type 23; HTTP payload is encrypted.

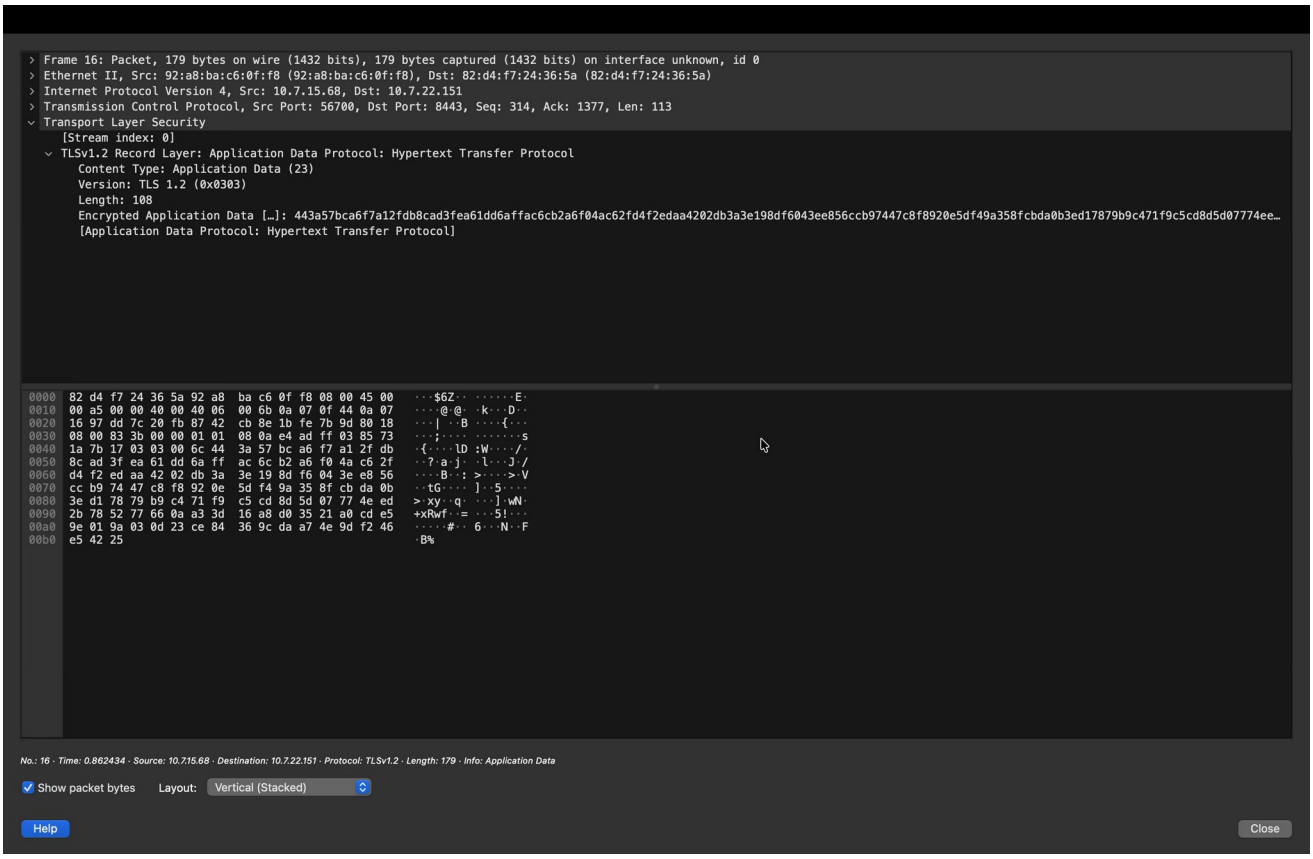


Figure E16. TLS record content type 23; HTTP payload is encrypted.

Evidence 17 Load balancing

Six HTTP/2 200 responses alternate A B A B A B.

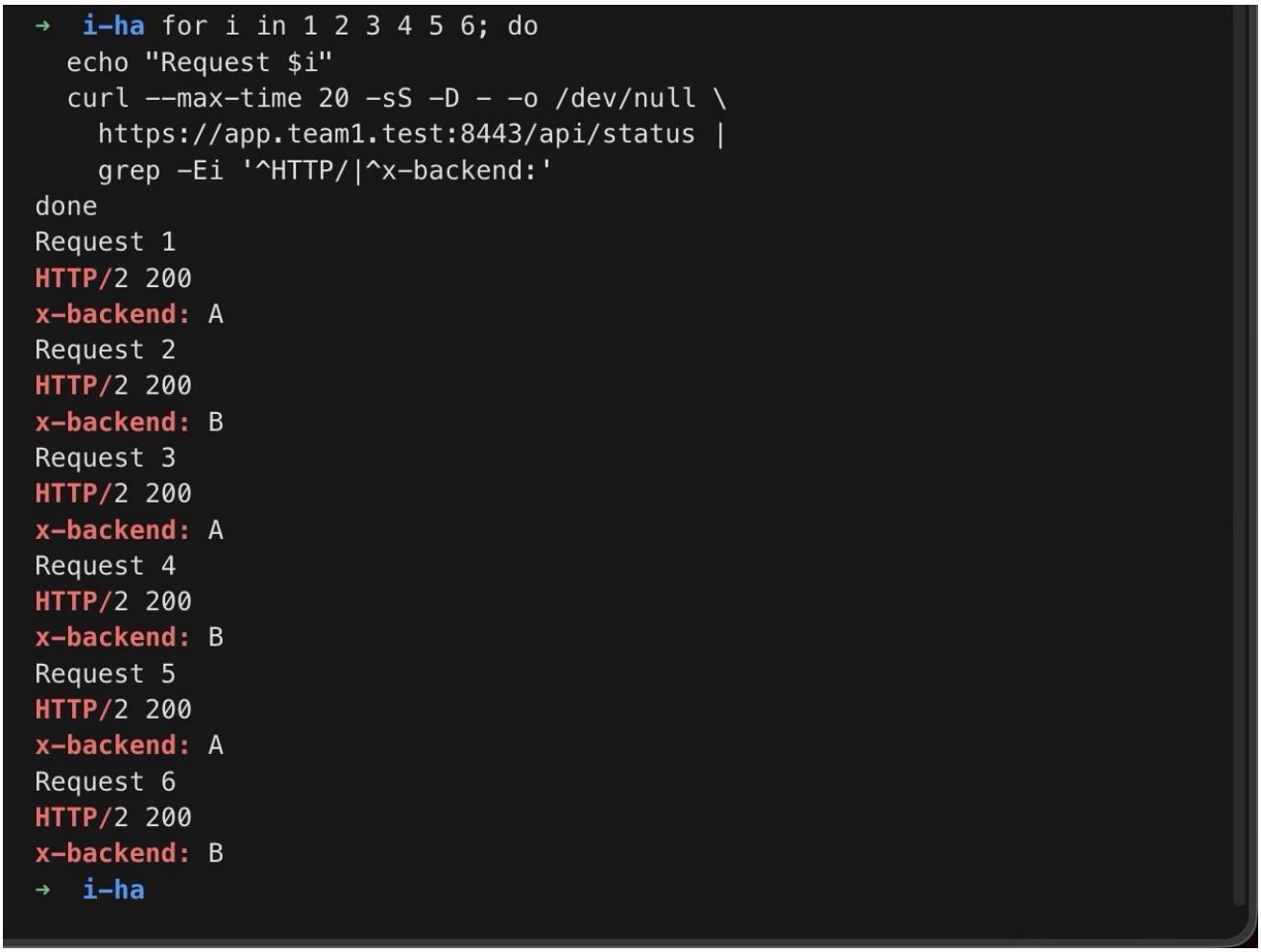


Figure E17. Six HTTP/2 200 responses alternate A B A B A B.

Evidence 18 Cacheable response

Backend A returns 200 with max-age=60 and an ETag.

```
→  i-ha curl --max-time 20 -i https://app.team1.test:8443/cache
HTTP/2 200
server: nginx/1.31.6
date: Sun, 04 Oct 2026 18:09:59 GMT
content-type: application/json
content-length: 73
x-backend: A
cache-control: public, max-age=60
etag: "18a8b57eb4f97ce2cde279d8685269a520bb718a79c03da046d874988903b098"
x-edge: Chaitanya

{"message":"Same cacheable representation on both backends","version":1}
→  i-ha
```

Figure E18. Backend A returns 200 with max-age=60 and an ETag.

Evidence 19 Conditional cache validation

Same ETag sent in If-None-Match; B returns 304 with no body.

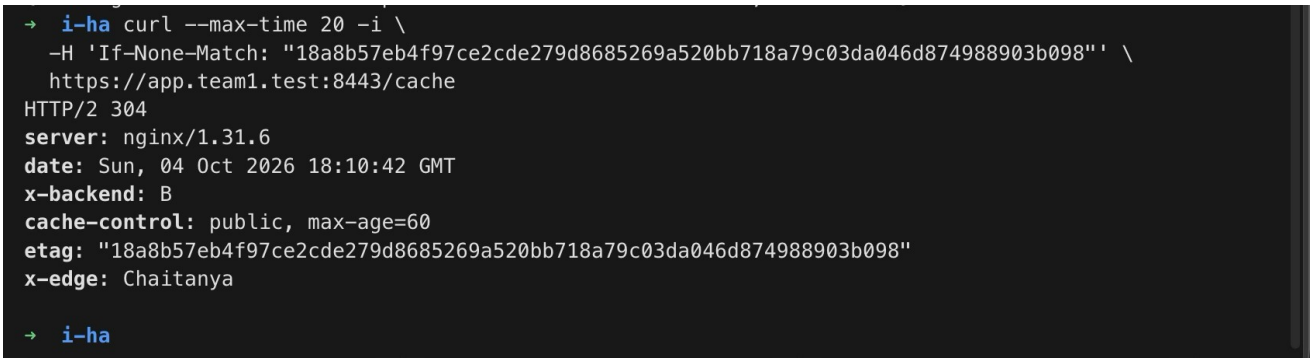


Figure E19. Same ETag sent in If-None-Match; B returns 304 with no body.

Evidence 20 One backend unavailable

After coordinated A shutdown, B serves HTTP/2 200.

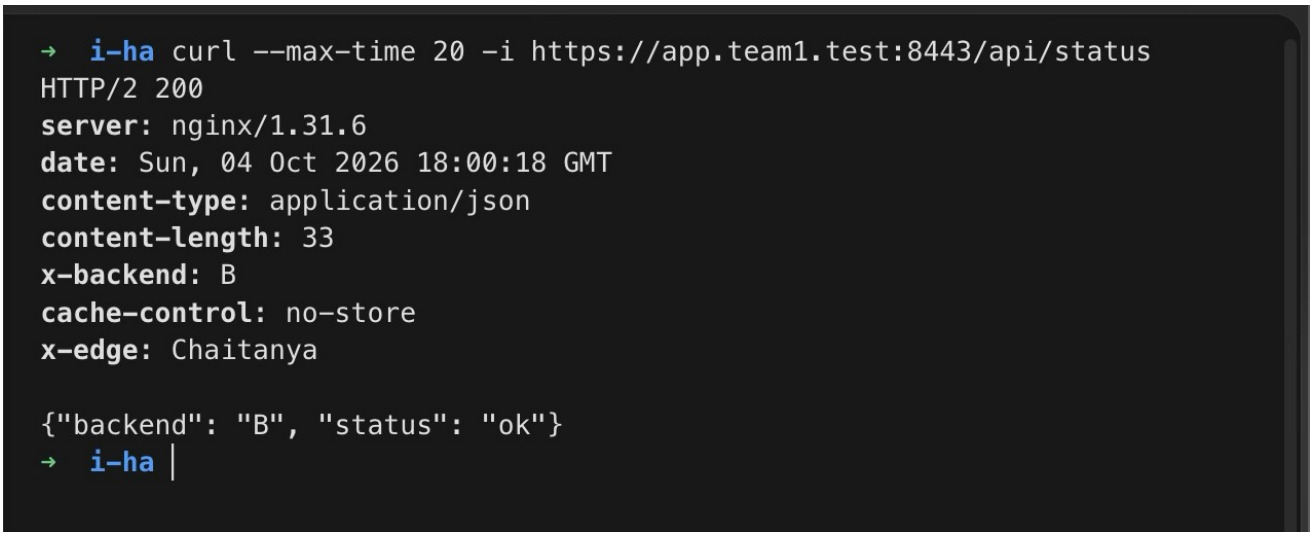


Figure E20. After coordinated A shutdown, B serves HTTP/2 200.

Evidence 21 Both backends unavailable

Observed HTTP/2 504. Logs show B refused connection then A timed out.

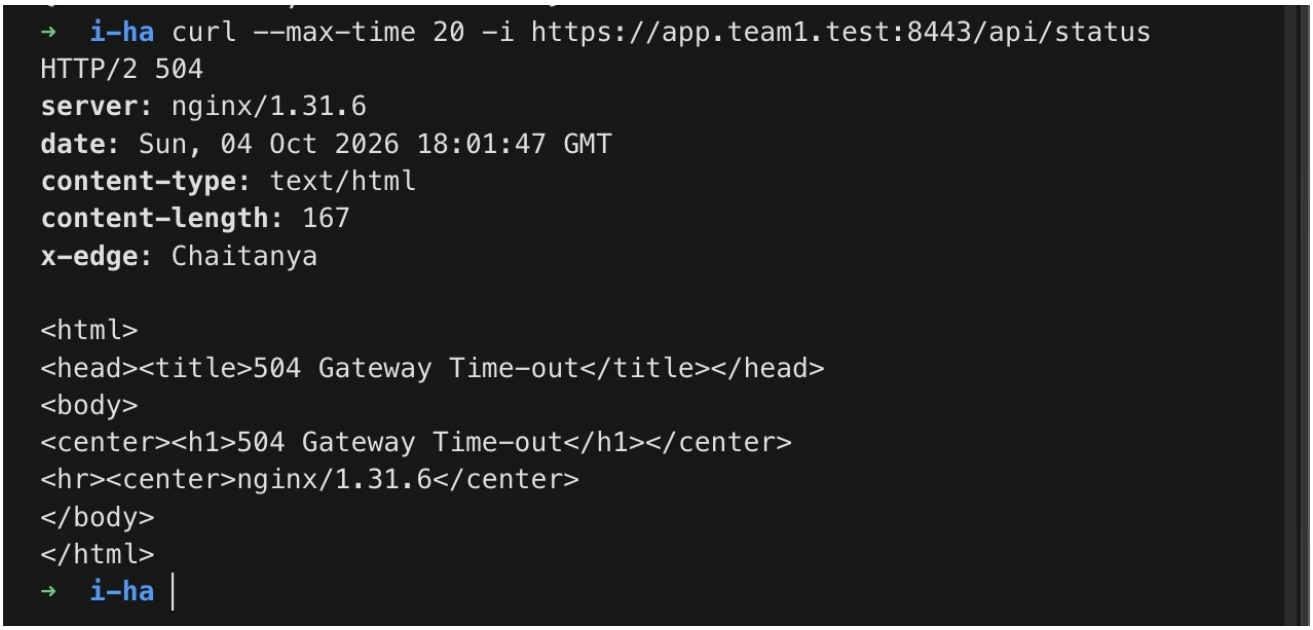


Figure E21. Observed HTTP/2 504. Logs show B refused connection then A timed out.

Evidence 22 Backend service recovery

B restores service; later A/B alternation confirms full recovery.



Figure E22. B restores service; later A/B alternation confirms full recovery.

Evidence 23 Wrong destination port

DNS succeeds, but TCP 8444 connection is refused.

```
➜  i-ha curl --connect-timeout 5 --max-time 8 -v https://app.team1.test:8444/api/status
* Host app.team1.test:8444 was resolved.
* IPv6: (none)
* IPv4: 10.7.22.151
*   Trying 10.7.22.151:8444...
* connect to 10.7.22.151 port 8444 from 10.7.15.68 port 57835 failed: Connection refused
* Failed to connect to app.team1.test port 8444 after 1610 ms: Couldn't connect to server
* Closing connection
curl: (7) Failed to connect to app.team1.test port 8444 after 1610 ms: Couldn't connect to server
➜  i-ha
```

Figure E23. DNS succeeds, but TCP 8444 connection is refused.

Evidence 24 Correct destination port

Returning to 8443 yields HTTP/2 200.

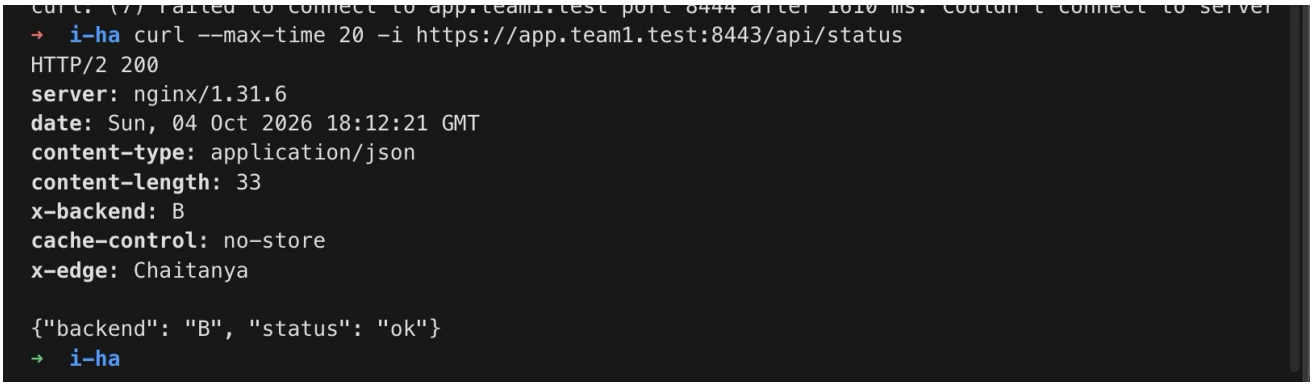


Figure E24. Returning to 8443 yields HTTP/2 200.

Evidence 25 Wrong DNS resolver

Configured resolver 10.7.2.89 yields DNS timeout; direct ping succeeds.

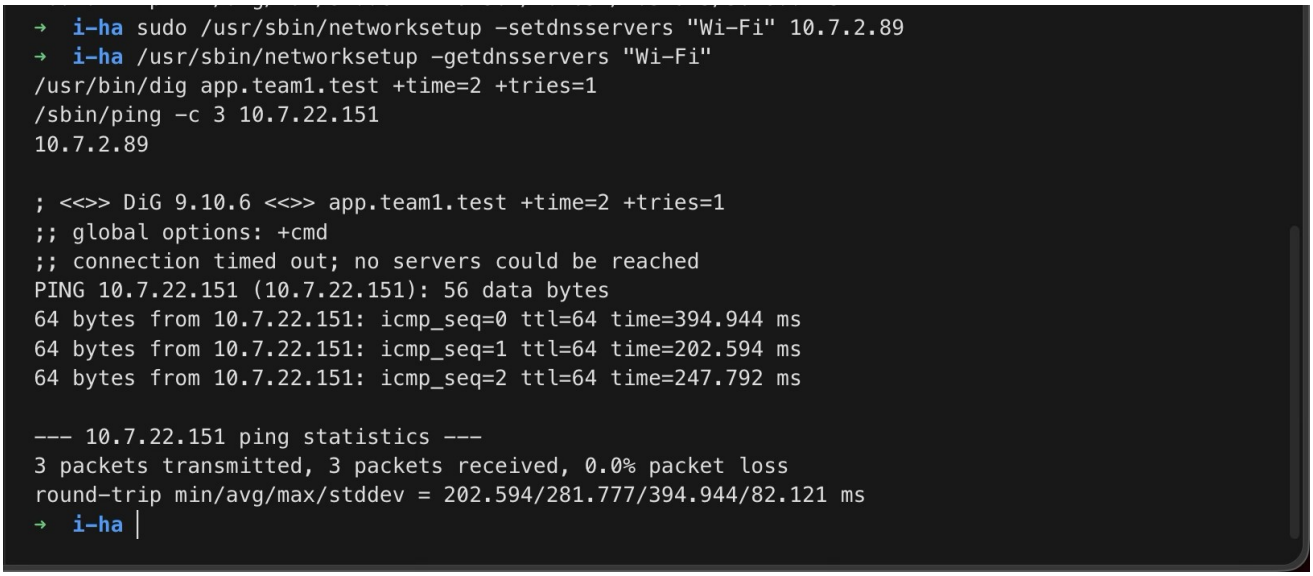


Figure E25. Configured resolver 10.7.2.89 yields DNS timeout; direct ping succeeds.

Evidence 26 DNS resolver recovery

Resolver restored to 10.7.15.68; correct DNS answer returns.

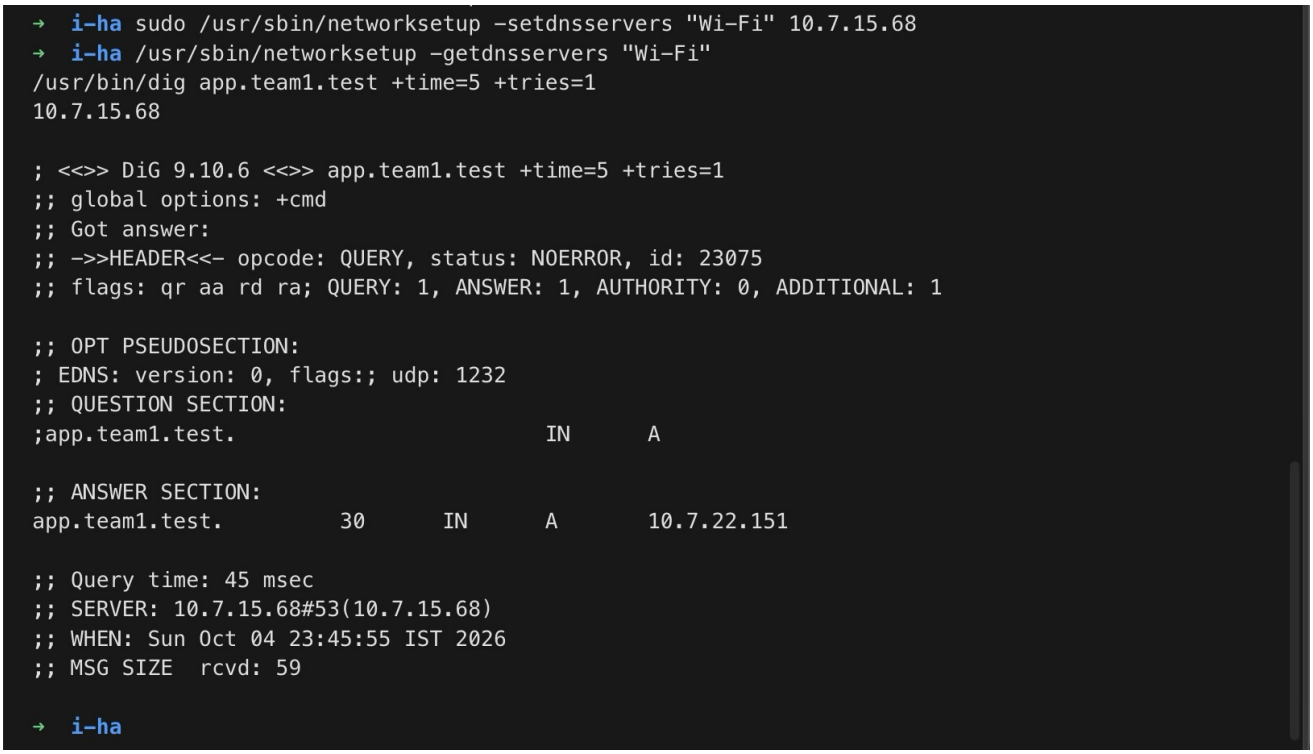


Figure E26. Resolver restored to 10.7.15.68; correct DNS answer returns.

Evidence 27 Wrong DNS record

DNS succeeds but returns 10.7.2.89, the wrong HTTPS destination.

```
→  i-ha /usr/bin/dig app.team1.test +time=5 +tries=1

; <<>> DiG 9.10.6 <<>> app.team1.test +time=5 +tries=1
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31362
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.team1.test.                        IN      A

;; ANSWER SECTION:
app.team1.test.         30      IN      A       10.7.2.89

;; Query time: 38 msec
;; SERVER: 10.7.15.68#53(10.7.15.68)
;; WHEN: Sun Oct 04 23:48:42 IST 2026
;; MSG SIZE  rcvd: 59

→  i-ha
```

Figure E27. DNS succeeds but returns 10.7.2.89, the wrong HTTPS destination.

Evidence 28 Wrong record connection failure

curl uses the incorrect address and TCP connection times out.

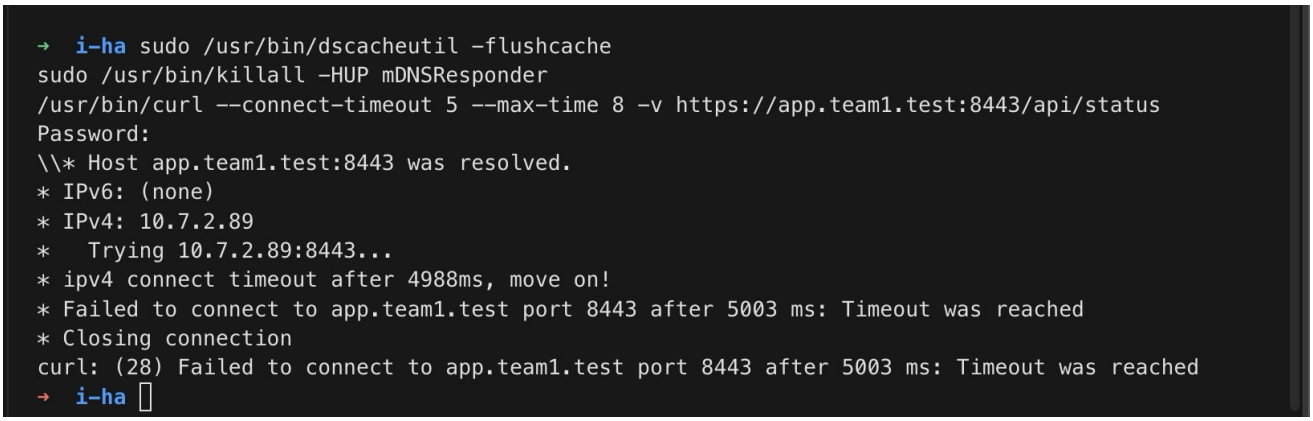


Figure E28. curl uses the incorrect address and TCP connection times out.

Evidence 29 DNS record recovery

Correct answer 10.7.22.151 and trusted HTTPS 200 from A.

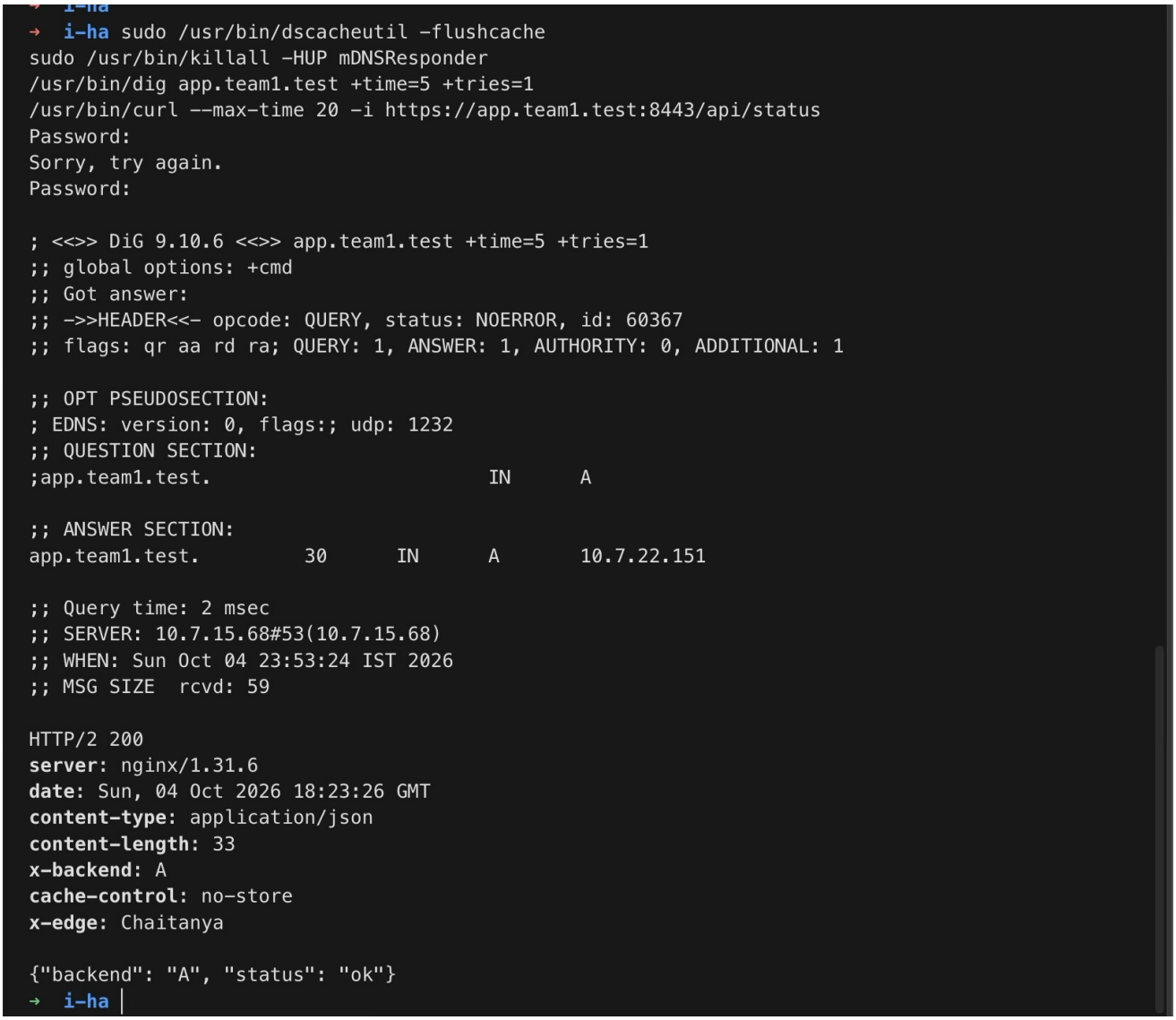


Figure E29. Correct answer 10.7.22.151 and trusted HTTPS 200 from A.

Evidence 30 Windows ping firewall rule

Project ICMP echo allow rule is enabled for the three team Macs.



Figure E30. Project ICMP echo allow rule is enabled for the three team Macs.